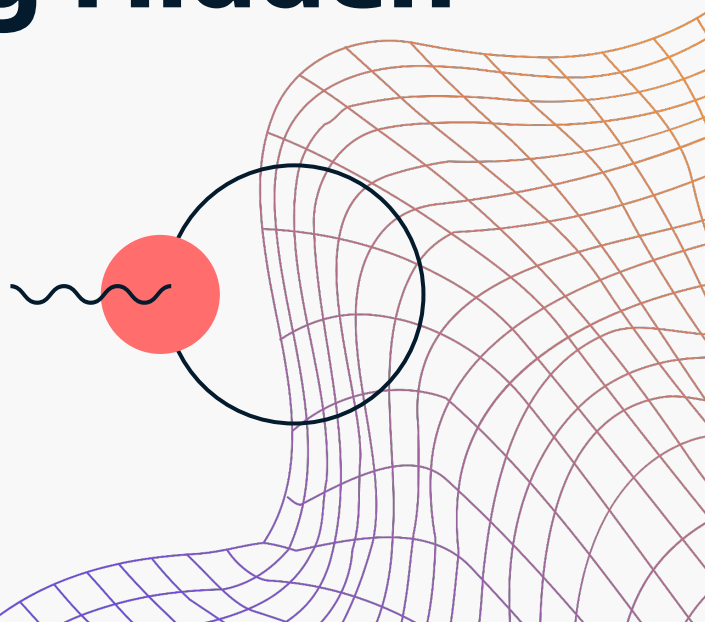
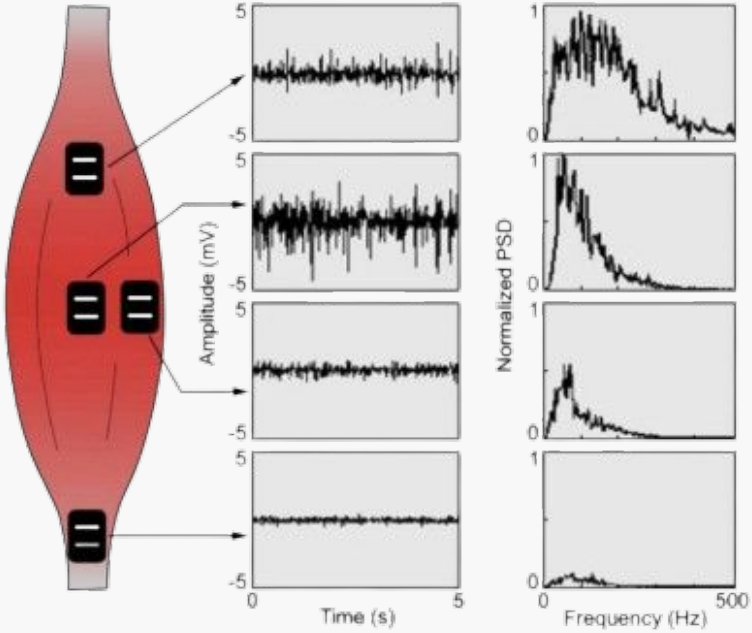
Electromyography Signal Classification Using Hidden Markov Models

Charith, Darian, Jacob



Introduction to EMG Waves

- Tiny electrical signal sent off when muscles contract
- Electrodes get placed on the skin
- Multiple sensors work together (sensor fusion)
- Prosthetic limbs
- Also used to test for diseases



Why improve EMG Signal Processing?

Prosthetic latency

- Delay time on moving prosthetics
- Ensure safety

Human Computer Interaction

- EMG controlled devices are on the rise
- Easier for those with disabilities

Rehabilitation

- Can be placed on rehabilitating athletes to check to see if muscles are failing

Literature Review

Mathematical Characterization Techniques

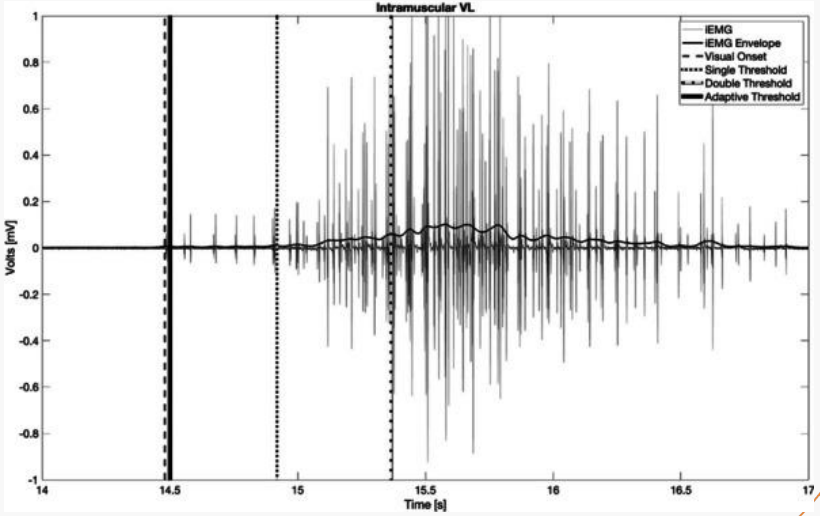
- Frequency & Time–Frequency Analysis (Fourier, Wavelets)
- Statistical & Higher-Order Statistics

Onset Detection

- Probabilistic Threshold
- Machine Learning

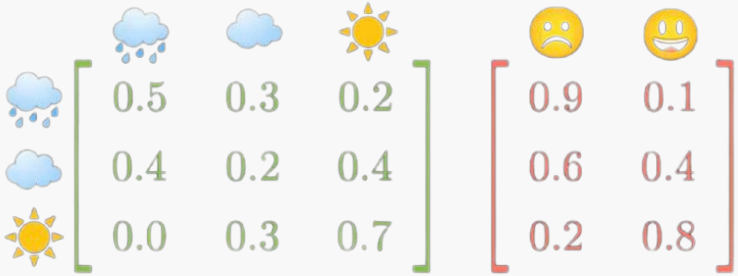
Computational Classification Methods

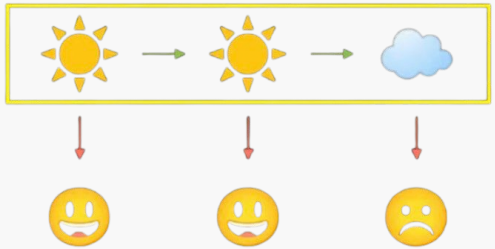
- Artificial Neural Networks
- Dynamic Neural Networks
- Fuzzy Logic Systems
- Genetic Algorithms

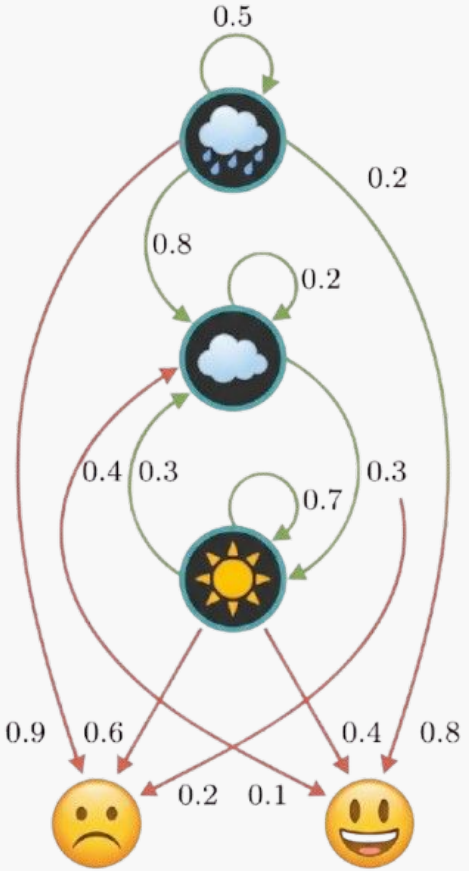


Hidden Markov Model

- HMM is a bridge that connect what we measure and what we want from the measurement
- HMM understands context and sequence







Gather Data

1. Used a **Myoware** sEMG sensor to collect data
2. Connect sensor to **Arduino** analog pin and use script to collect serial data
3. Use a **Python** script to record the data in 1 second segments for each gesture, and place into a data collection folder
4. Ensure the following criteria for clean data:
    a. High Sampling Frequency
    b. Large enough window size
    c. Onset detection
    d. Consistent stages in each sample



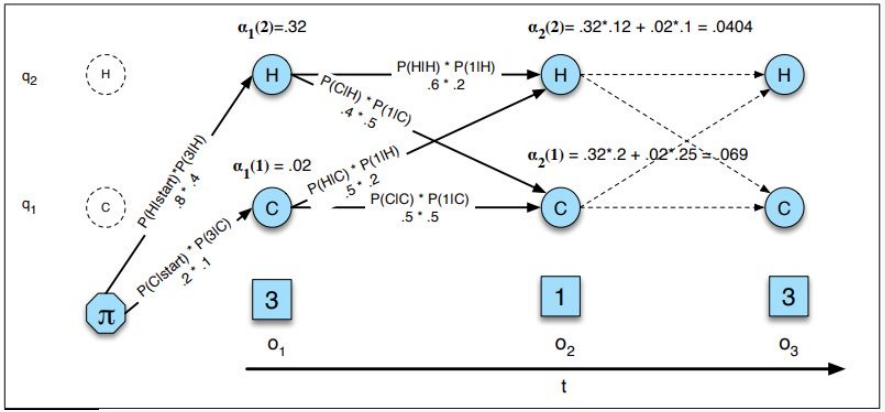
Implementation Workflow

1. We treat our signal like a sequence of observations
2. Train the Hidden Markov Model Transition Matrix and Emission Matrix using a 4 state automaton and the signal. (Baum-Welch Algorithm)
3. Find out where the HMM switches states using the viterbi algorithm
4. Fit an HMM to each gesture and use the likelihood algorithm
5. The highest likelihood match corresponds to the gesture

Knowns	Unknown	Algorithm Used
Observation sequence O, number of hidden states	Transition matrix A, emission matrix B	Baum–Welch Algorithm
HMM parameters $\lambda=(A,B)$, observation sequence O	Optimal hidden state sequence Q	Viterbi Algorithm
HMM parameters $\lambda=(A,B)$, observation sequence O	Likelihood $P(O\mid\lambda)$	Forward Algorithm

Forward Algorithm

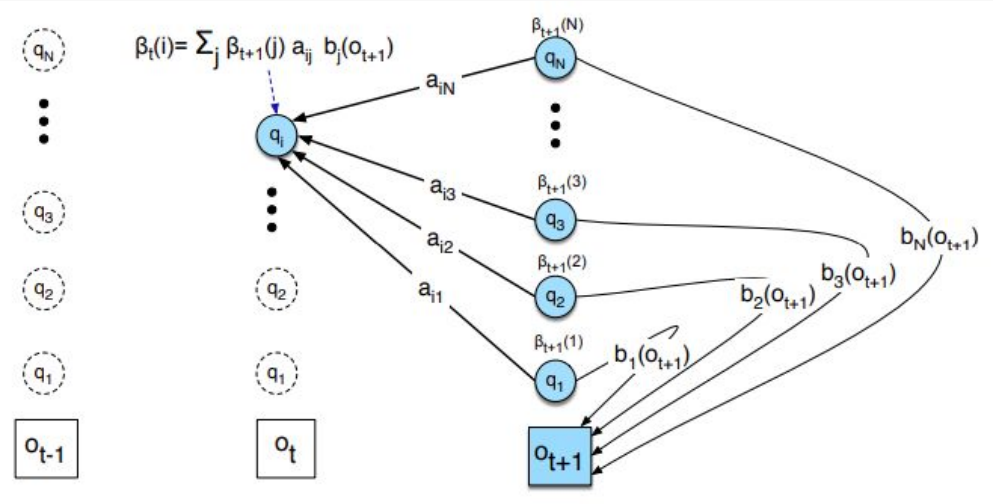
- $\alpha_t(j)$ -> Probability of being in state j after t observations.
- Calculated recursively, which makes it computationally efficient
- An example of dynamic programming
- $O(N^2T)$ time complexity
- Used to calculate the likelihood that a sequence occurs on a given automaton



$$\alpha_t(j) = P(o_1, o_2 \ldots o_t, q_t = j|\lambda) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t)$$

Backward Algorithm

- $\beta_t(j)$ -> probability of seeing the observations from time t+1 to the end, given state j at time t.



$$\beta_t(i) = P(o_{t+1}, o_{t+2} \ldots o_T | q_t = i, \lambda)$$

Baum Welch (Forward Backward) Algorithm

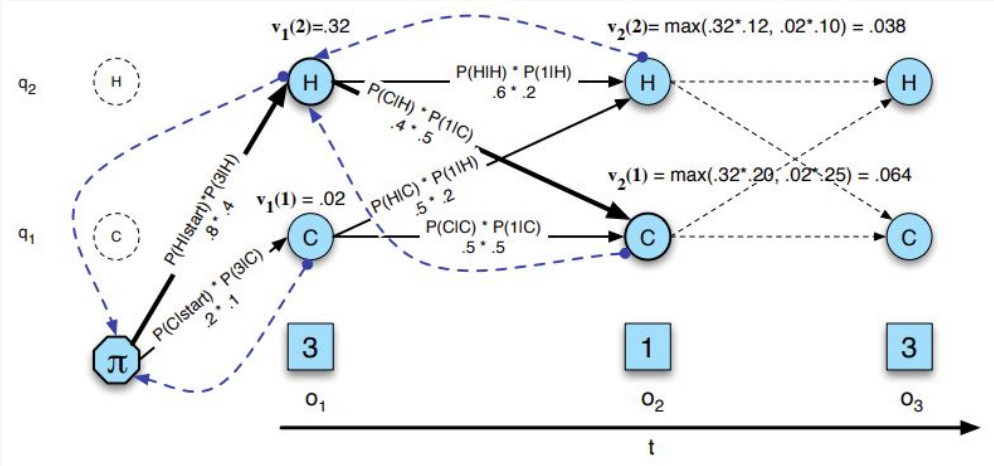
- Given the automaton and an observation sequence, learn the HMM parameters A and B
- Iterative algorithm that runs until it converges upon the “trained” A and B matrices
- Integrates both the forward and backward algorithms
- Provided with an initial guess
- This is how we trained our HMM

$$\hat{a}_{ij} = \frac{\text{expected number of transitions from state } i \text{ to state } j}{\text{expected number of transitions from state } i}$$

$$\hat{b}_j(v_k) = \frac{\text{expected number of times in state } j \text{ and observing symbol } v_k}{\text{expected number of times in state } j}$$

Viterbi Algorithm

- Given a sequence of observations O and the automaton, determine the most probable sequence of states Q = q1q2q3…qT.
- You are looking for the maximum path probability to get to a certain state
- Need to keep track of where you have been using backpointers.
- This is how we do onset detection!



CNN vs HMM Implementation

Neural Network

Accuracy: ~60%

- Discriminative → Direct mapping from sEMG window to gesture label
- More Data Hungry
- More sensitive to noise → Not as resilient to rapid switching
- Less setup

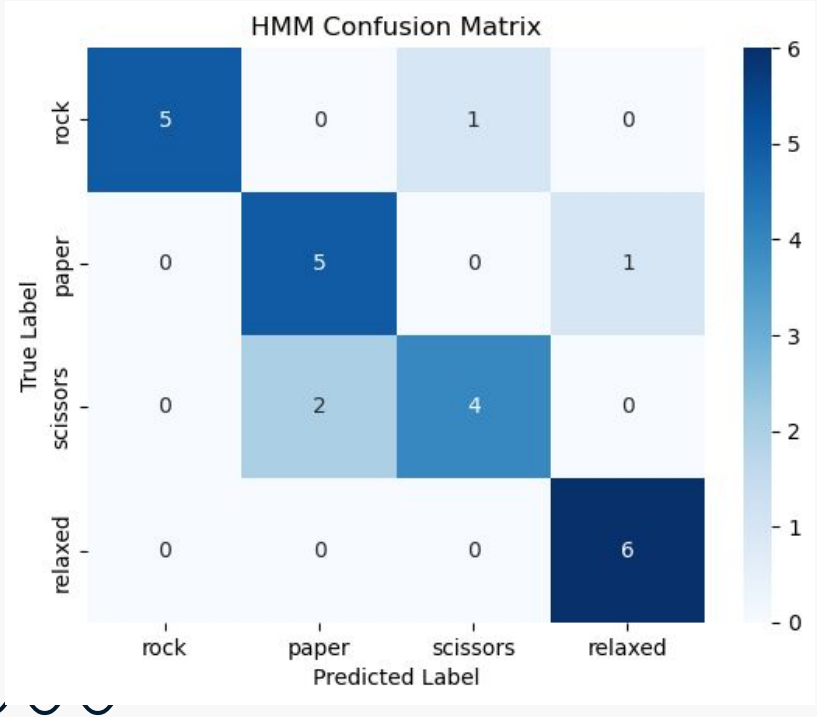
Hidden Markov Model

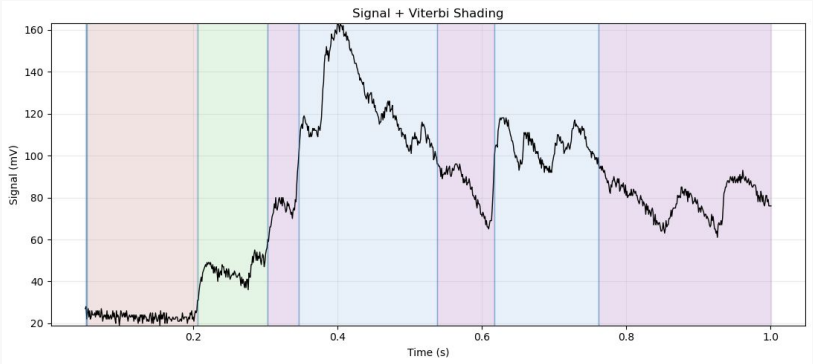
Accuracy: ~75%

- Generative → How could a gesture generate the observed sequence
- Can work with less data, but more sensitive
- More interpretable (why did a classification happen)

Both models were trained on 4 gestures, each with 20 samples.

Results


HMM Confusion Matrix
True Label
rock
paper
scissors
relaxed
5
0
1
0
0
5
0
1
0
2
4
0
0
0
0
6
rock
paper
scissors
relaxed
Predicted Label
6
5
4
3
2
1
0


Signal + Viterbi Shading
Signal (mV)
160
140
120
100
80
60
40
20
0.2
0.4
0.6
0.8
1.0
Time (s)

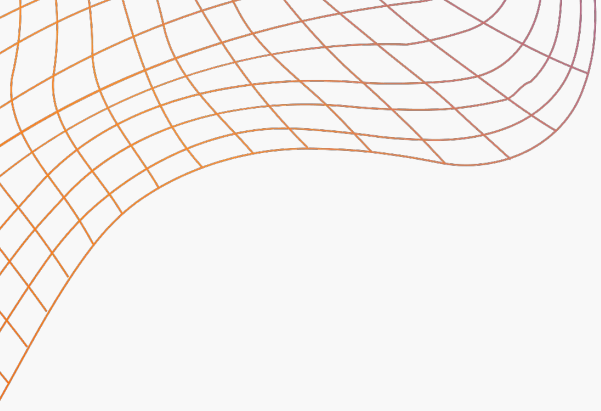
Future Work

Analyze Viterbi Plots

By analyzing the Viterbi Plots, we can determine which states were misclassified and look at any patterns that show up in our data. This can help us further tune our model.

Data Analysis/Collection

Our model could have been done a lot of good by simply using more data with better collection techniques. E.g. more channels, better sampling, better filtering, etc.



Thank you!

Questions?